1. Decision and scope

Mathematical memo — 13 November 2026. Exact-real semantics; no training or GPU execution.

The slot-store update admits a compact affine scan **relative to fixed routing, selection, gate magnitudes, and liveness decisions**. Its affine summaries compose associatively. Sequential execution, chunk execution, and ordered reduction trees give the same state. Hard gate decisions and state-dependent death checks obstruct an unconditional affine representation on the original state coordinates.

Lean witnesses: `EmenderMemory.token_conditional_scan; token_chunked_scan; MemoryAlgebra.ScanTree.ordered_tree_equiv; MemoryAlgebra.Obstructions.hard_gate_not_affine_on_unit_interval; state_dependent_death_not_affine`

There are two distinct meanings of "arbitrary memory operator." Any transition on any state type has a faithful linear action after encoding complete states as basis vectors. That general construction is proved, but has **no compact representation or work guarantee**. For the practical store, a stronger result holds: fixed-decision updates remain diagonal affine maps on the original numeric coordinates. The generic interface also instantiates the existing GDN2 update definition.

Lean witnesses: `MemoryAlgebra.LiftedMemory.encode_injective; lift_correct; lift_comp; arbitrary_family_scan; MemoryAlgebra.DiagonalOperator.toAffine_thenOp; GDNMemoryOperator.embed_correct`

This memo uses short namespaces: **A** = `MemoryAlgebra.AffineOperator`, **M** = `MemoryAlgebra`, **E** = `EmenderMemory`, **R** = `RecurrentResourceFormalism`, **O** = `MemoryAlgebra.Obstructions`. Every result box names Lean witnesses. Definitions are identified as definitions. Source correspondences and systems costs are explicitly assumptions, inspection notes, or estimates; they are not kernel-checked Python equivalence or performance results.

Obligation	Delivered formal surface
Algebra / P1	Affine summaries; update-family interface; a semantic predicate for independent programs; universal lifted embedding.
Five operations	Keys, values, strength and age; overwrite, residual write, multiplicative pop/decay; separate Boolean death bookkeeping.
Conditional scan	A schedule-indexed semantic property, valid-trace bridges, and explicit gate/death counterexamples.
Scan equivalence	Sequence, arbitrary chunks, every prefix, ordered trees, and indexed suffix propagation of injections.
Activity	One content-write slot per token; exact sparse reconstruction; full-state activity at most B, sometimes exactly B.

Inspection note. The original `RecurrentResourceFormalism.emender` describes E88/NDM with a `tanh` state update. It is not the X30 slot store. Its historical `scanCompatible = false` remains intact. The new `slotStoreSemantics` and semantic companion give the slot-store statement without reclassifying nonlinear E88. The former P1 gap is addressed at the level of supplied independent program updates, not as a theorem that an entire multilayer training system scales.

2. Algebra, generality, and P1

Definitions (A). Over a real module S, an `AffineOperator S` is a pair (L, b) with $L : S \to S$ linear and $b \in S$. Its action is $s \mapsto L(s) + b$. Multiplication is chronological: $a \star c$ means "apply a first, then c." Thus

$$(L_1, b_1) \star (L_2, b_2) = (L_2 \circ L_1, L_2(b_1) + b_2).$$

The identity is $(I, 0)$. Associativity follows by composing the linear parts and distributing L_3 over the two injection terms. Both parenthesizations have linear part $L_3 \circ L_2 \circ L_1$ and injection $L_3(L_2(b_1)) + L_3(b_2) + b_3$. Lean installs a `Monoid` instance, stronger than the required semigroup. Elementary operation labels need not be closed under composition; their **summaries** are.

Lean witnesses: `A.thenOp_assoc; A.apply_one; A.apply_mul; A.linear_mul; A.injection_mul`

Definition (M.MemoryOperator). The interface is parameterized by an elementary operator type and a state module. It supplies `step`, `embed`, and the law `step op s = (embed op).apply s` for every state. This is a semantic law, not a Boolean flag. Instantiating the class requires proving affinity in those state coordinates. The elementary Emender and token-decision instances discharge the law against independently defined arithmetic; the GDN instance reuses the established `gdnStep_eq_affine` theorem.

Lean witnesses: `E.step_eq_diagonal; E.token_summary_correct; GDNMemoryOperator.embed_correct; M.memoryOperator_fold`

For the GDN2 parameter record, the embedded transition is left multiplication by `splitGatedTransition 1 k b * decay`, with injection `outerKV k (e97WriteValue w v)`. This uses the same definition as the existing GDN bridge, including the order of pre-decay. The resulting fold has an affine summary for each fixed gate/value/decay schedule.

Lean witnesses: `GDNMemoryOperator.gdn2_conditional_scan; GDNMemoryOperator.gdn2_fold_scan`

Arbitrary transitions, with an explicit change of state space. For a state type X, encode x as the finitely supported vector e_x. The linear push-forward of $f : X \to X$ maps e_x to $e_{f(x)}$ and combines coefficients when images collide. The encoding is injective; push-forward respects composition and the complete fold. This permits arbitrary hard routing and nonlinear updates algebraically on the lifted space. **Size assumption/limitation:** a basis indexed by full states may be prohibitively large or infinite; no efficient representation follows from the existence theorem. The compact scan claims below concern the original store only.

Lean witnesses: `M.LiftedMemory.encode_injective; lift_correct; lift_comp; arbitrary_family_scan`

P1 predicate. `IsMultiProgrammedRecurrentStack family update` requires at least two programs and states that program p's output depends on its own previous state through the supplied family. For any family, pointwise application meets this predicate; projecting a whole run onto one program equals that program's own run. GDN2 and the slot store are named specializations. Affinity is not required here. **Scope assumption:** inter-layer inputs are already supplied; no independence of their computation, training, tiling, or communication is claimed.

Lean witnesses: `M.every_update_family_multiProgrammed; M.independent_fold_projection; R.slotStore_multiProgrammed; R.gdn2_multiProgrammed`

3. Store coordinates and the five operations

Definitions (E.Field, E.Store, E.State). There are B slots. A numeric row has K key coordinates, V value coordinates, strength r, and age a. Write $d = K + V$ for content width. Boolean liveness is carried separately. All numeric coordinates, including age, are modeled over the reals. Integrality of age, nonnegativity, gate ranges, and integer overflow are not refinement theorems here.

The direct `step` definition and the independent `scale` / `injection` encoding agree on every state. The following table gives selected-row arithmetic; every unmentioned coordinate is preserved. Indices and all displayed numbers are fixed arguments of the operator.

Operation	Selected-row numeric rule	Bookkeeping
APPEND	$k_{j'} = k_t$, $v_{j'} = g_t v_t$, $r_{j'} = 1$, $a_{j'} = 0$.	Marks exactly j live.
CORRECT	If enabled, $v_{j'} = v_t$; otherwise identity.	No allocation; key, strength and age retained.
MERGE	$v_{j'} = v_j + \beta_t(v_t - v_j)$.	No allocation; key, strength and age retained.
POP	$r_{j'} = r_j(1 - p_t)$ if a target is present.	Death at threshold; numeric data retained.
DECAY / AGE	For every live i: $r_{i'} = \gamma_t r_i$, $a_{i'} = a_i + 1$.	Death after decay; dead rows remain frozen.

Lean witnesses: `E.step_eq_diagonal; append_overwrites; correct_full_overwrite; correct_disabled; merge_delta_rule; pop_strength; decay_live_strength_age; decay_dead_frozen`

"Full overwrite" for CORRECT means the full **value vector**, as in the supplied implementation. It does not rewrite the key or reset age. MERGE has the residual form for a general fixed β. With $\beta = 1$, MERGE equals enabled CORRECT on the entire numeric store. This is the specialization used by binary module gates.

Lean witnesses: `E.correct_preserves_key; merge_preserves_key; merge_one_eq_correct`

Source inspection and refinement assumptions. The reference equations were read in `experiments/expressivity_tasks/x30_tape_cell/naive_reference.py`, especially lines 278–407 and the module's binary gate construction at 504–506, and in `cell.py`, especially 439–469, 514–552 and 686–693. The lean model follows these equations over exact reals; equivalence to executed Python, fp32 correction tricks, timer wheels, or replay gradients is not proved. Both module entry points use detached binary write enables. The generic scan APIs can still receive a nonbinary magnitude, which the MERGE family represents.

Routing assumptions. A recorded APPEND target is assumed to be free when the architecture selects it. CORRECT/MERGE targets are assumed to be the selected matched slots. The proof does not implement youngest-slot choice, adjacency matching, residual thresholds, allocator order, or pressure bookkeeping. This keeps the precise numeric operator semantics separate from the unproved producer of its decisions. Allocation itself is explicit: any live transition from dead to live must be an APPEND, and APPEND can revive only the selected slot.

Lean witnesses: `E.only_append_allocates; E.append_allocates_only_selected`

4. Death, token order, and diagonal decomposition

Definition (E.freeMask). For a live row, death is the comparison $r_i \leq \tau$, including equality. POP first multiplies strength, then performs this comparison. DECAY first multiplies live strengths and increments their ages, then performs the comparison. A live row survives exactly when the updated strength is strictly above threshold. Death changes the Boolean flag; it does not zero the numeric row.

Lean witnesses: `E.freeMask_iff; E.pop_death_iff; E.decay_death_iff; E.decay_dead_frozen`

Definition (E.TokenDecision). A token records a pop target/magnitude, pop death mask, at most one write, the live mask before decay, decay rate, and decay death mask. Numeric execution is POP, WRITE, DECAY/AGE. The sequential operation list and the direct token definition agree. READ is pre-pop and pre-write in the source implementation (inspection note); the read function itself is outside the update theorem.

Lean witnesses: `E.token_step_eq_fold; E.token_summary_correct`

For a fixed operator, define a diagonal array D and an injection array b. APPEND has zero diagonal entries on the selected row. Enabled CORRECT has zero entries on its value coordinates. MERGE has $1 - \beta$ there. POP and DECAY supply the strength multipliers. Other retained coordinates have diagonal entry one. Thus the diagonal is not restricted to decay/pop: it also represents overwrite and residual erasure. Ages contribute an additive one on each live row.

Lean witnesses: `E.step_eq_diagonal; E.step_decomposition`

Definition (M.slotInjection). $(e_j \otimes \delta)(i, f)$ is $\delta(f)$ when $i = j$, and zero otherwise. Lean proves this is the one-hot scalar times the row vector. With A the injection's slot support, the exact decomposition is

$$s' = D \odot s + \sum_{j \in A} e_j \otimes b_j.$$

Here b_j is the injection row, not the post-update value. In MERGE its value component is βv_t while D carries the $1 - \beta$ coefficient. The expression does not freeze the state-dependent residual as though it were a constant. "Rank-one injection" means this outer-product form; a zero row has rank zero. The Lean proof uses one-hot functions rather than a matrix-rank API.

Lean witnesses: `M.slotInjection_eq_onehot; M.diagonal_slot_decomposition; E.step_decomposition; E.token_decomposition`

Diagonal operators are closed under composition:

$$(D_1, b_1) \star (D_2, b_2) = (D_2 \odot D_1, D_2 \odot b_1 + b_2).$$

This representation has two arrays regardless of the number of composed steps. The token summary is obtained by composing the elementary diagonals; its affine embedding agrees with the direct token update. **Representation estimate:** storing these dense arrays costs a constant multiple of the numeric state size. This is an array-size interpretation, not a memory-allocation benchmark.

Lean witnesses: `M.DiagonalOperator.toAffine_thenOp; toAffine_composeAll; E.token_summary_correct`

5. Conditional compatibility and obstructions

Definition (M.ScanCompatibleRelativeTo). A supplied schedule is compatible when each scheduled decision has an affine representation valid for every numeric state. The representation may depend on the decision; it must not depend on the state argument being scanned. The class instances establish this property, and the conditional existence theorem produces a summary for the entire schedule.

Lean witnesses: `M.memoryOperator_scanCompatible; M.conditional_scan_exists; E.token_conditional_scan; R.slotStore_scan_relative`

The semantic taxonomy companion carries an actual update family. Its `scanCompatibleRelativeTo` property is indexed by the schedule, unlike the legacy Boolean. A detached selector can depend on prior state in the forward pass: detachment removes an autodiff dependency, not a mathematical dependency. **Interface assumption:** every decision, numerical gate, append residual/gate input, and live/death mask needed by the state scan is fixed before evaluating that scan.

There are two verified replay bridges. `TraceMatches` requires the selected operators to equal the choices made along a particular routed trajectory; under that hypothesis the routed run equals the scan. `TokenDecision.ValidFor` requires correct pop-death, post-write-live, and decay-death masks. Under that hypothesis, replay equals a token evaluation that recomputes threshold deaths. Neither predicate proves that a router can produce all decisions in parallel.

Lean witnesses: `M.routedRun_eq_replay; M.routedRun_eq_scan_of_trace; E.replay_eq_checked`

Inspection note. `cell.py` replay retains the detached error used to form the append gate. The naive scan can recompute a detached residual even when supplied a routing trace. A routing trace alone is therefore not automatically the fixed numeric gate schedule assumed here; the gate values or their state-derived error inputs must also be fixed. No gradient-equivalence claim is made.

Named gate obstruction. Take old value zero, new value one, and enable CORRECT iff $g > \frac{1}{2}$. Its result is $h(g) = 1$ above the threshold and zero otherwise, exactly the value of `hardCorrect` on the zero store. Any affine map obeys $f(x+y) + f(0) = f(x) + f(y)$. Yet

$$h(1) + h(0) = 1 \neq 0 = h\left(\frac{1}{2}\right) + h\left(\frac{1}{2}\right).$$

All three gates lie in the permitted interval $[0, 1]$. Hence no affine map of the gate agrees even on that interval. With the Boolean decision fixed, CORRECT is still affine in the state.

Lean witnesses: `A.affine_identity; O.hard_correct_eq_gate; O.hard_gate_witness; O.hard_gate_not_affine_on_unit_interval`

Named state obstruction. Check a live strength x for death at $\frac{1}{2}$, then apply decay $\frac{1}{2}$ using the resulting live mask. The numeric result is $f(x) = x$ for $x \leq \frac{1}{2}$, and $\frac{x}{2}$ otherwise. Therefore $f(1) + f(0) = \frac{1}{2}$ while $f\left(\frac{1}{2}\right) + f\left(\frac{1}{2}\right) = 1$. Recomputing death masks defeats affinity even when all decay magnitudes are fixed. Function composition remains associative; the failed property is closure in the compact affine state family, not associativity itself.

Lean witnesses: `O.deathThenDecay_formula; O.death_threshold_witness; O.state_dependent_death_not_affine; A.thenOp_assoc`

6. Sequence, chunks, prefixes, and scan trees

Let $a_0, \ldots, a_{T-1}$ be affine operators in temporal order. **Definition (A):** `composeAll []` is the identity, and `composeAll (a :: as)` is `a * composeAll as`. Induction on the list proves

$$\mathrm{foldl}(s \mapsto a_t(s), s_0) = (a_0 \star \ldots \star a_{T-1})(s_0).$$

The head operator acts first, and the induction hypothesis handles the tail at the updated starting state. The result is connected explicitly to the existing function-level `CompositionalDepthDuality.composeAll` theorem. The extra result here is closure in an algebraic representation; generic function composition alone supplies no compact scan implementation.

Lean witnesses: `A.foldl_eq_composeAll; A.composeAll_eq_function_composition; M.memoryOperator_fold; E.token_fold_scan`

For any two lists u, v, the summary of u followed by v is the summary of u multiplied by the summary of v. Induction moves the cut across the first list using associativity. Lists may be empty. There is no commutativity assumption, and operators must never be permuted in time.

Lean witnesses: `A.composeAll_append`

Partition a token schedule into arbitrary chunks $c_0, \ldots, c_{N-1}$. Let A_j be the summary of chunk c_j. Evaluating each chunk sequentially from the preceding state equals evaluating the chunk summaries. Flattening the chunks and running the original fold gives the same final state. The chunk sizes can differ; there is no divisibility requirement and empty chunks are identities.

Lean witnesses: `A.composeAll_flatten; A.chunked_fold_equiv; E.token_chunked_scan`

For a prefix inside a chunk, compose the summaries of the preceding complete chunks, apply that prefix summary to the resulting entry state, and obtain the same state as the original fold through that token. This licenses a two-stage implementation: obtain chunk entry states from a prefix scan of summaries, then evaluate local prefixes using those entry states. **Algorithm assumption:** the implementation materializes those summaries and entry states faithfully.

Lean witnesses: `A.prefix_scan_equiv; A.chunk_prefix_equiv`

The formal `ScanTree` datatype contains identity leaves, operator leaves, and ordered binary joins. Structural induction proves that a tree's summary equals `composeAll` of its leaves. This covers any balanced parenthesization once its leaf order is the temporal order. It does not prove a particular GPU scan kernel, barrier protocol, or asymptotic scheduler bound.

Lean witnesses: `M.ScanTree.summary_eq_composeAll; M.ScanTree.ordered_tree_equiv`

Relation to prior proofs (inspection note). The implementation reuses the fold/composition idiom of `foldl_eq_composeAll`, the affine composition law of `gdnStateFold_affine_compose`, and the time-order convention documented by `WeightedAutomata.foldl_matMul_eq_rev`. The earlier E97 and E1H composition theorems remain valid, but do not themselves imply an efficient affine scan for those nonlinear recurrences. No earlier Boolean classification theorem was used as a substitute for a semantic equality.

7. Injection propagation and activity

Write $a_i = (L_i, b_i)$. For a prefix of length n, define the ordered linear product $P_n = L_{n-1} \circ \ldots \circ L_0$, with empty product I. The explicit prefix formula is

$$s_n = P_n(s_0) + \sum_{i=0}^{n-1} (L_{n-1} \circ \ldots \circ L_{i+1})(b_i).$$

The injection at time i is propagated through exactly the **subsequent** linear parts. The final injection has an empty propagation product. Lean defines `linearProduct` and `propagatedInjections` recursively and also proves the indexed sum using `drop (i+1)`. The prefix theorem uses `take n`; indices beyond the end truncate safely. No inverse is required, so zero erasures and full overwrites are covered. This is preferable to formulas that divide by prefix products.

Lean witnesses: `A.composeAll_linear; A.composeAll_injection; A.injection_propagation; A.injection_eq_suffix_sum; A.prefix_product_injection_formula`

Definition (M.slotSupport). A slot belongs to the support of an injection if some field in that row is nonzero. Any one-hot slot injection has support contained in its singleton target, hence cardinality at most one. Given any certified activity superset, summing the row injections over that set reconstructs the entire injection exactly. This is a semantic justification for skipping all other injection rows.

Lean witnesses: `M.slotSupport_injection_subset; slotSupport_injection_card; sparse_reconstruction; E.sparse_step`

Diagonal multiplication cannot spread injection support into a new slot. The support of a sum is contained in the union, allowing cancellation. Consequently the injection support after diagonal summary composition is contained in the union of the two input supports. These facts justify propagation and combination of activity sets; they do not imply that a long chunk retains just one active slot.

Lean witnesses: `M.diagonal_support_subset; M.support_add_subset; M.diagonal_composition_support`

For an elementary operation, `activeSlots` is its write singleton, the empty set for POP, or the set of live rows for DECAY/AGE. Exact injection support lies within this set, and its cardinality is at most B. Projecting onto key/value content gives a sharper bound of one, both for elementary operations and for a whole token with at most one write.

Lean witnesses: `E.injection_support_subset; E.injection_activity_bound; E.content_injection_support_le_one; E.token_activity_bound; E.token_content_support_le_one`

Necessary qualification. Full-state activity is not always one slot. When all rows are live, age increments inject one into every age coordinate, so the support is exactly all B slots. A two-slot witness refutes the claimed bound of one. The required decomposition is still exact, with these global age increments included among its slot injections. **Implementation estimate:** timestamps or a separate global age offset may avoid explicit dense age work, but equivalence of such a representation is not proved here. A content-sparsity theorem must not be reported as a full-state constant-activity theorem.

Lean witnesses: `E.age_injection_dense; E.age_activity_not_one_slot`

8. Cost model and implementation consequences

All asymptotic statements on this page are estimates under the assumptions below, not Lean complexity or benchmark theorems. The algebraic license is `token_chunked_scan`, `chunk_prefix_equiv`, and `ordered_tree_equiv`; they prove equal states, not speed.

Let T be tokens, $C \geq 1$ tokens per chunk, and $N = \lceil \frac{T}{C} \rceil$. Assume fixed decisions, enough workers for independent chunks, constant-depth abstract summary composition, and no communication or bandwidth bottleneck. A token-by-token recurrence has $O(T)$ sequential update stages. After chunk summaries are formed, the serial chain of **inter-chunk** applications has $O(N) = O(\frac{T}{C})$ stages. This is the precise scope of the requested "$O(T)$ to $O(\frac{T}{C})$" depth statement.

It is not a complete end-to-end depth bound by itself. If local summaries and local replays each take $O(C)$ serial stages but chunks operate concurrently, a serial inter-chunk pass gives total staged depth $O(C + N)$. A balanced prefix scan of summaries instead gives the ideal estimate $O(C + \log N)$. If C denotes the **number of chunks** rather than chunk length, the corresponding estimate is $O(\frac{T}{C} + \log C)$. These conventions must not be mixed. If decisions must first be generated by a sequential state-dependent run, that stage can retain $O(T)$ depth.

> **No claimed unconditional training speedup.** A replay scan can shorten the numeric state path while decision generation, reads, layer dependencies, and memory bandwidth still dominate. None of those bottlenecks is removed by the associativity theorem.

Representation/work estimates. A diagonal summary stores a multiplier and an injection for each numeric coordinate. Dense composition therefore takes $O(B(d + 2))$ arithmetic and storage per summary, with no dense $B(d + 2)$-square matrix required. General affine/GDN summaries may have different costs; the abstract `MemoryOperator` interface supplies no universal work bound. The universal lifted construction is particularly not an efficiency argument.

Read/write volume assumption. Hold token inputs, selected operations, and the number of read/write queries fixed. A chunk scan changes evaluation order and dependencies, not the per-token content that those operations must read or write. It therefore does not eliminate the original per-token read/write volume. For a dense content-addressed read, an implementation may still examine all live keys/values; a one-slot content write still touches its row. These are source-level cost estimates, not a proved lower bound on all possible implementations.

Summary construction, prefix storage, local replay, and communication can add traffic. "Unchanged per-token read/write volume" must not be interpreted as "unchanged total bytes transferred" or "unchanged total work." The age support counterexample additionally prevents deriving constant full-store traffic from the content activity bound.

Implementation contract. Precompute or retain the complete fixed-decision schedule, construct each diagonal summary, preserve chronological order, compute chunk entries, and evaluate needed local prefixes. Apply pre-write reads to the appropriate prefix states. Retain both death masks and the gate inputs required for replay. Verify a proposed kernel against the direct recurrence separately: this memo supplies the exact-real target and the assumptions that such a test must exercise, not a tested runtime implementation.

9. Coverage limits and validation record

What this does NOT cover. The following are explicit assumptions or open refinement tasks, not proved properties of the production system.

- **Decisions inside the scan:** argmax, adjacency/youngest matching, residual comparisons, allocation order, and death detection are not computed by the compact affine scan. Supplied decisions must satisfy the recorded-trace hypotheses. The named hard-gate/death counterexamples rule out the proposed unconditional original-coordinate affinity, not every possible representation.
- **Soft versus hard routing:** fixed soft coefficients can sometimes form an affine operator, but no general soft-router instance is supplied. State-dependent softmax reads and gate networks are not in the theorem. The hard-selector counterexample does not establish a theorem about all soft routing families.
- **Numerical precision:** Lean uses exact reals. Reassociation can change fp32, bf16 or mixed-precision results, rounding, death times, and gradients. There is no bitwise-parity, stability, error-bound, or backward-pass theorem. Fixed masks avoid recomputing branches in the model; they do not prove numerical accuracy.
- **Whole model and management:** usage counters, read outputs, lazy timestamps, timer wheels, allocator structures, pressure windows, optimizer state, integer-age refinements, and cross-layer networks are outside the formal state. Existing E88/NDM nonlinear semantics are not reclassified as affine.
- **Performance and compactness:** no GPU scan was implemented or benchmarked, and no training, scheduler, or multi-device scaling claim is certified. All costs on the preceding page are estimates with stated assumptions.

Assumption boundary. The arithmetic theorems hold for all real numeric stores and supplied operators; they do not need well-formed gates or nonnegative strengths. To interpret a schedule as a native architectural trajectory, assume the source-to-Lean translation, valid targets, identical token parameters, correct gate values, and trace consistency. `ValidFor` and `TraceMatches` make parts of that boundary explicit hypotheses, rather than hidden global axioms.

Proof inventory. `OPERATOR_ALGEBRA_STATUS.md` maps every deliverable to its files and principal theorems. `OPERATOR_ALGEBRA_AUDIT.json` contains the complete machine-readable source-theorem inventory, source locations, and transitive axiom sets. The reproducible audit rejects nonstandard axioms and placeholder proof dependencies across all declarations of the eight new modules.

Validation note. The baseline root built successfully at source commit `5cdb2f12668802096e11b2a040c30f3169e0c55e`. The delivered root imports the new modules through `ElmanProofs.PaperCore`; `lake build` passed (2207 jobs). The dependency audit passed for 626 declarations and 92 source theorems, and the trusted import-closure check passed for 25 project source files. Exact output, source hashes, compilation details, and the validated source commit are recorded in the status document and `build_logs/operator_algebra_*`. The audit found no new axiom declarations or unfinished proofs. The machine summary lists `sorryCount: 0` and `sorryReasons: []`. Standard Lean foundations (propositional extensionality, classical choice, and quotient soundness) are allowed and reported per theorem.

Commands to reproduce from `formal/lean`:

```
lake build
scripts/check_paper_core.sh
lake env lean scripts/audit_operator_algebra.lean
typst compile EMENDER_OPERATOR_ALGEBRA.typ EMENDER_OPERATOR_ALGEBRA.pdf
```

The unresolved implementation work is to produce and validate the fixed schedule, choose a representation for global age work, and test a numerical scan against the native recurrence. The checked mathematical result is the conditional equivalence, with explicit negative results at its boundary.